

Projet Next.js — Plateforme de galeries photographiques privées

Contexte

Vous travaillez pour une jeune entreprise qui souhaite créer une plateforme destinée aux photographes indépendants.

Aujourd'hui, beaucoup de photographes transmettent leurs photos à leurs clients à l'aide de services génériques : WeTransfer, Google Drive, Dropbox, dossiers partagés, etc.

Ces solutions permettent de transmettre des fichiers, mais répondent mal au besoin métier du photographe.

Après un mariage, un shooting professionnel, une séance familiale ou un événement, le photographe souhaite pouvoir :

- organiser ses clients ;
- créer une galerie dédiée à chaque prestation ;
- publier facilement les photographies correspondantes ;
- donner un accès privé à son client ;
- permettre au client de parcourir agréablement plusieurs dizaines ou centaines de photos ;
- permettre au client de sélectionner ses photographies préférées ;
- retrouver ensuite cette sélection depuis son espace professionnel.

Votre mission consiste à **concevoir et développer cette plateforme avec Next.js**.

1. Objectif du projet

Vous disposez de **16 heures** pour réaliser une application fonctionnelle suffisamment aboutie pour pouvoir être présentée comme un véritable produit.

L'objectif n'est donc pas uniquement :

« faire fonctionner des pages ».

Vous devrez également travailler :

- l'expérience utilisateur ;
- les performances ;
- le responsive ;
- les états de chargement ;
- les erreurs ;
- la sécurité ;
- l'architecture de l'application ;
- la qualité du code ;
- les interactions ;
- la cohérence graphique.

Vous êtes libres concernant l'identité visuelle et l'interface du produit.

En revanche, plusieurs **contraintes fonctionnelles et techniques sont imposées**.

2. Données et backend

Un serveur **PocketBase** vous est fourni.

Il contient les données nécessaires au fonctionnement de l'application et expose une API permettant notamment de récupérer et modifier :

- des utilisateurs ;
- des clients ;
- des galeries ;
- des photographies ;
- les sélections de photographies.

Vous n'avez pas à développer le backend.

Votre application Next.js devra communiquer avec cette API.

Des identifiants de démonstration vous seront fournis.

3. Les utilisateurs de la plateforme

Le produit possède deux types d'utilisation très différents.

Photographe

Le photographe possède un compte et accède à un espace professionnel authentifié.

Il peut administrer ses clients, ses galeries et consulter les sélections effectuées.

Client du photographe

Le client final ne possède pas nécessairement de compte.

Le photographe lui transmet :

- un lien vers sa galerie ;
- un code privé.

Exemple :

```
https://application.fr/gallery/a8df73
```

Code d'accès :
4829

Après avoir renseigné le bon code, le client peut accéder à sa galerie.

4. Authentification du photographe

L'application doit proposer une page de connexion.

Exemple :

```
/login
```

Après authentification, le photographe accède à son espace professionnel.

Exemple :

```
/dashboard
```

Les pages du dashboard ne doivent pas être accessibles à un utilisateur non authentifié.

Une simple dissimulation des liens ou des boutons dans l'interface **n'est pas considérée comme une protection**.

L'application doit réellement vérifier que l'utilisateur possède les droits nécessaires.

Le photographe doit également pouvoir se déconnecter.

5. Dashboard

Une fois connecté, le photographe arrive sur son tableau de bord.

Vous êtes libres concernant son contenu exact.

Il devra néanmoins fournir une vision utile de son activité.

Vous pourriez par exemple afficher :

- nombre de clients ;
- nombre de galeries ;
- galeries récemment créées ;
- galeries en attente de sélection ;
- sélections récemment terminées ;
- nombre total de photographies ;
- derniers clients.

L'objectif n'est pas d'afficher artificiellement un maximum d'informations.

Le dashboard doit être **simple, lisible et utile**.

6. Gestion des clients

Le photographe doit pouvoir consulter ses clients.

Exemple :

```
/dashboard/clients
```

Pour chaque client, les informations disponibles pourront notamment être :

- nom ;
- prénom ;
- adresse e-mail ;
- téléphone ;
- date de création ;
- galeries associées.

Une page dédiée doit permettre de consulter un client.

Exemple :

```
/dashboard/clients/[id]
```

Cette page pourra également permettre de retrouver toutes ses galeries.

7. Gestion des galeries

Le photographe doit pouvoir consulter ses galeries.

Exemple :

```
/dashboard/galleries
```

Une galerie correspond généralement à une prestation photographique.

Exemples :

```
Mariage Alice & Thomas  
Portraits entreprise — ACME  
Séance famille Martin  
Anniversaire Léa
```

Pour chaque galerie, l'interface devra permettre de retrouver facilement des informations comme :

- son nom ;
- son client ;
- sa date ;
- son nombre de photos ;
- son état ;
- l'avancement de la sélection.

Exemple :

```
Mariage Alice & Thomas  
  
186 photos  
Client : Alice Martin  
Sélection : 42 / 50  
  
État :  
Sélection en cours
```

8. États d'une galerie

Une galerie possède un cycle de vie.

Votre interface devra représenter clairement son état.

Par exemple :

```
Brouillon
Accessible au client
Sélection en cours
Sélection terminée
Archivée
```

Les états disponibles dépendront des données fournies par l'API.

L'état doit être compréhensible rapidement depuis le dashboard ou la liste des galeries.

9. Page d'administration d'une galerie

Une page doit permettre au photographe d'administrer une galerie.

Exemple :

```
/dashboard/galleries/[id]
```

Elle devra notamment présenter :

- les informations de la galerie ;
- le client associé ;
- les photographies ;
- son état ;
- le nombre de photographies sélectionnées ;
- la sélection du client.

Le photographe doit pouvoir facilement identifier les photographies choisies par son client.

10. Création et modification

Le photographe doit pouvoir effectuer des opérations de modification sur les données.

Au minimum, votre application devra permettre plusieurs opérations parmi :

- créer une galerie ;
- modifier une galerie ;
- supprimer une galerie ;
- créer un client ;
- modifier un client ;
- ajouter des photographies ;
- supprimer une photographie ;
- modifier l'état d'une galerie.

Les formulaires doivent gérer correctement :

- les validations ;
- les erreurs ;
- l'état pendant l'envoi ;
- le succès de l'opération.

Une double soumission involontaire ne doit par exemple pas créer plusieurs fois la même ressource.

11. Accès à une galerie client

La partie client de l'application est accessible depuis une URL dédiée.

Exemple :

/gallery/[id]

Une galerie privée ne doit pas immédiatement afficher ses photographies.

Le client doit auparavant renseigner son code d'accès.

Exemple :

Cette galerie est privée.

Code d'accès :

[]

[Accéder à ma galerie]

Si le code est incorrect, une erreur compréhensible doit être affichée.

Si le code est correct, l'utilisateur doit pouvoir continuer à naviguer dans sa galerie sans devoir retaper son code à chaque interaction.

12. Consultation des photographies

Une fois authentifié sur sa galerie, le client doit pouvoir parcourir les photographies.

L'expérience de consultation constitue une partie importante du produit.

L'interface doit donc être adaptée à un usage photographique.

Vous êtes libres de concevoir par exemple :

- une grille ;
- un masonry layout ;
- différentes tailles d'aperçu ;
- une lightbox ;
- une navigation plein écran ;
- une navigation au clavier ;
- un affichage progressif.

Le résultat doit fonctionner correctement sur ordinateur et sur mobile.

13. Sélection de photographies

Le client doit pouvoir sélectionner ses photographies préférées.

Exemple :



puis :



La sélection doit être enregistrée.

Le client doit pouvoir identifier immédiatement les photos déjà sélectionnées.

L'interface doit également afficher l'avancement de sa sélection.

Par exemple :

32 / 50 photos sélectionnées

Le nombre maximal dépendra éventuellement de la galerie.



14. Finalisation d'une sélection

Lorsqu'il a terminé, le client doit pouvoir valider sa sélection.

Par exemple :

[Valider ma sélection]

Une confirmation doit être demandée avant une opération ayant des conséquences importantes.

Après validation, le photographe doit pouvoir constater depuis son dashboard que le client a terminé.

Exemple :

Mariage Alice & Thomas

50 / 50 photos sélectionnées

✓ Sélection terminée

Vous devrez réfléchir au comportement de l'application après validation :

- la sélection peut-elle encore être modifiée ?
- faut-il afficher un message particulier ?
- quelles actions restent disponibles ?

Votre choix doit être cohérent.



15. Performance : contrainte particulière

Une galerie photographique peut contenir **plusieurs centaines de photographies**.

Votre application doit être pensée pour cette réalité.

Une solution consistant simplement à charger simultanément 200 photographies originales de plusieurs mégaoctets n'est pas acceptable.

Vous devrez réfléchir notamment :

- au poids des images ;
- à leur taille d'affichage ;
- au chargement différé ;
- aux miniatures ;
- à l'utilisation des outils d'optimisation proposés par Next.js ;
- au nombre de ressources chargées ;
- au JavaScript envoyé au navigateur ;
- au comportement sur une connexion mobile.

Une galerie importante doit rester utilisable.

////////////////////////////////////

16. Responsive

Le produit doit être totalement utilisable :

- sur ordinateur ;
- sur tablette ;
- sur smartphone.

La galerie client sera très probablement consultée depuis un téléphone.

Le responsive ne doit donc pas être considéré comme une amélioration facultative ajoutée à la fin du projet.

Il fait partie du produit.

////////////////////////////////////

17. Gestion du chargement

Une application utilisant une API externe ne répond pas toujours instantanément.

Votre application doit gérer cette situation.

L'utilisateur ne doit pas se retrouver devant une page vide en attendant une réponse.

Vous devrez prévoir des interfaces adaptées :

- skeletons ;
- loaders ;
- interfaces intermédiaires ;
- désactivation temporaire d'un bouton ;

- feedback pendant une opération.

Vous devrez notamment utiliser lorsque cela est pertinent les mécanismes proposés par l'App Router de Next.js.



18. Gestion des erreurs

L'application doit correctement gérer les erreurs.

Quelques exemples :

- identifiant de galerie inconnu ;
- client inexistant ;
- API indisponible ;
- erreur pendant une modification ;
- mauvais code d'accès ;
- utilisateur non autorisé ;
- ressource supprimée ;
- formulaire invalide.

Une erreur ne doit pas simplement produire une page blanche ou une exception incompréhensible.

Vous devrez notamment exploiter les mécanismes appropriés de Next.js comme les Error Boundaries de route et la gestion des ressources inexistantes.



19. Architecture Next.js

Vous devez utiliser l'**App Router** de Next.js.

Le Pages Router ne doit pas être utilisé pour ce projet.

Next.js propose aujourd'hui l'App Router comme architecture permettant notamment d'exploiter les Server Components.

L'arborescence devra exploiter intelligemment :

```
app/
```

ainsi que :

- les pages ;

- les layouts ;
- les routes dynamiques ;
- les différents segments de l'application.

Une organisation comme celle-ci pourrait par exemple émerger :

```
app/  
  
  login/  
  
  dashboard/  
    layout.tsx  
    page.tsx  
  
    clients/  
      page.tsx  
      [id]/  
        page.tsx  
  
    galleries/  
      page.tsx  
      new/  
        page.tsx  
      [id]/  
        page.tsx  
  
    gallery/  
      [id]/  
        page.tsx
```

Cette structure est uniquement illustrative.

Vous êtes libres d'en proposer une meilleure.

20. Server Components et Client Components

Une attention particulière sera portée à la séparation entre :

- Server Components ;
- Client Components.

Avec l'App Router, les composants sont rendus côté serveur par défaut. Les Client Components sont nécessaires lorsqu'une fonctionnalité dépend notamment de l'état du

navigateur, de certains événements ou d'API exclusivement disponibles côté client.

Vous ne devez donc pas transformer toute votre application en Client Components.

Par exemple :

```
"use client"
```

ne doit pas être placé automatiquement au début de chaque fichier.

Pour chaque Client Component important, vous devez être capable d'expliquer :

■ Pourquoi ce composant doit-il être exécuté dans le navigateur ?

À l'inverse, vous devez être capable d'expliquer pourquoi certaines opérations ont intérêt à rester côté serveur.

21. Récupération des données

Vous devrez réfléchir à l'endroit où sont récupérées les données PocketBase.

Toutes les données n'ont pas nécessairement besoin d'être récupérées depuis le navigateur.

Par exemple, une page de dashboard peut récupérer ses informations côté serveur puis transmettre uniquement ce qui est nécessaire aux composants interactifs.

Un schéma comme celui-ci :

```
Navigateur
  ↓
useEffect()
  ↓
fetch()
  ↓
API
```

ne doit pas être votre réponse automatique à tous les problèmes.

22. Mutations

Vous aurez également besoin de créer et modifier des données.

Vous devrez choisir les outils appropriés selon les situations.

L'utilisation des **Server Actions** est notamment attendue lorsqu'elle est pertinente. Next.js permet d'utiliser ces actions pour traiter des mutations et des formulaires côté serveur.

Vous devez être capable d'expliquer vos choix.

23. Routes dynamiques

Le projet doit exploiter des routes dynamiques.

Par exemple :

```
/dashboard/clients/[id]

/dashboard/galleries/[id]

/gallery/[id]
```

Il ne doit pas être nécessaire de créer manuellement une page différente pour chaque ressource.

24. Layouts

Votre application devra exploiter les layouts de Next.js.

Par exemple, toutes les pages :

```
/dashboard/*
```

pourraient partager :

- une navigation ;
- une sidebar ;
- les informations du photographe ;
- une action de déconnexion.

La partie galerie client ne doit en revanche pas nécessairement utiliser cette interface.

Vous devrez donc organiser correctement les différentes zones de l'application.

25. Sécurité

Plusieurs données manipulées sont privées.

Votre application devra donc respecter quelques principes élémentaires.

En particulier :

- les identifiants sensibles ne doivent pas être intégrés directement dans le code envoyé au navigateur ;
- une page privée ne doit pas devenir accessible uniquement parce que quelqu'un connaît son URL ;
- masquer un bouton n'empêche pas une opération d'être exécutée ;
- les droits doivent être vérifiés avant d'accéder à une ressource ;
- les variables d'environnement doivent être utilisées correctement ;
- une galerie privée ne doit pas devenir publique accidentellement.

Vous devrez être capables d'expliquer la différence entre :

■ cacher une fonctionnalité dans l'interface

et :

■ interdire réellement son utilisation.

26. Design

Aucune maquette ne vous est fournie.

Vous êtes responsables de la conception de l'interface.

Le produit doit posséder une identité graphique cohérente.

Vous êtes libres d'utiliser :

- CSS ;
- CSS Modules ;
- Tailwind CSS ;
- une bibliothèque de composants ;
- vos propres composants.

La qualité graphique sera prise en compte, mais elle ne doit pas masquer un produit techniquement incomplet.

Un design extrêmement travaillé avec une application non fonctionnelle ne répond pas au sujet.

27. Expérience utilisateur

Pensez votre application comme un produit qui pourrait réellement être utilisé.

Quelques questions à vous poser :

L'utilisateur comprend-il immédiatement où il se trouve ?

Une action importante possède-t-elle un retour visuel ?

Que se passe-t-il lorsqu'une opération prend deux secondes ?

Que se passe-t-il si elle échoue ?

Une suppression accidentelle est-elle facilement possible ?

Peut-on utiliser la galerie confortablement sur téléphone ?

Les boutons ont-ils des intitulés compréhensibles ?

Un client peu à l'aise avec l'informatique comprend-il comment sélectionner ses photos ?

28. Accessibilité

L'interface devra respecter les bonnes pratiques fondamentales d'accessibilité.

Soyez notamment attentifs :

- à la structure HTML ;
- aux labels des formulaires ;
- à la navigation clavier ;
- aux boutons ;
- aux contrastes ;
- aux textes alternatifs ;
- aux focus ;
- aux messages d'erreur.

Une interface visuellement réussie mais inutilisable au clavier est considérée comme incomplète.

29. Qualité du code

Votre projet devra être organisé de manière cohérente.

On évitera notamment :

- des composants de plusieurs centaines de lignes mélangeant toutes les responsabilités ;
- la duplication massive de code ;
- des appels API dispersés sans organisation ;
- des noms de variables incompréhensibles ;
- du code mort ;
- des `any` utilisés systématiquement ;
- des `"use client"` ajoutés sans réflexion.

Vous devez être capables de retrouver rapidement :

- l'accès aux données ;
- les composants UI ;
- les composants métier ;
- les actions ;
- les types ;
- les mécanismes d'authentification.

30. TypeScript

Le projet doit être développé en **TypeScript**.

Les principales données métiers devront être correctement typées.

Par exemple :

```
Gallery
Photo
Customer
User
Selection
```

Les types doivent aider au développement et non simplement être contournés avec :

```
any
```

31. États à ne pas oublier

Un produit ne possède pas uniquement son état idéal.

Pour chaque écran important, pensez au minimum aux différents cas :

Données présentes

24 galeries trouvées

Liste vide

Vous n'avez encore créé aucune galerie.

Chargement

Chargement...

Erreur

Impossible de récupérer les galeries.

Ressource inexistante

Cette galerie n'existe pas.

Ces situations font partie du produit.



32. Contraintes Next.js obligatoires

Votre projet devra démontrer l'utilisation pertinente des concepts suivants :

- App Router ;
- layouts ;
- routes dynamiques ;
- Server Components ;
- Client Components ;
- récupération de données côté serveur ;
- interactions côté client ;

- gestion des formulaires ;
- mutations ;
- au moins une Server Action pertinente ;
- états de chargement ;
- gestion des erreurs ;
- gestion des ressources inexistantes ;
- optimisation des images ;
- authentification ;
- contrôle d'accès ;
- TypeScript.

L'objectif n'est pas de cocher artificiellement chaque élément.

Ils doivent être utilisés lorsqu'ils répondent à un problème réel du produit.

////////////////////////////////////

33. Ce qui est libre

Vous êtes libres concernant :

- le nom du produit ;
- son logo ;
- son identité graphique ;
- les couleurs ;
- la typographie ;
- l'organisation précise du dashboard ;
- la présentation des photographies ;
- le système de navigation ;
- la disposition des pages ;
- les animations ;
- les composants ;
- les fonctionnalités supplémentaires.

Deux groupes travaillant à partir de la même API peuvent donc produire deux applications complètement différentes.

35. Documentation

La documentation officielle Next.js fait partie de vos outils de travail.

Vous êtes encouragés à l'utiliser.

Vous n'avez pas vocation à connaître chaque fonction du framework par cœur.

Un développeur doit être capable :

1. d'identifier son problème ;
2. de rechercher la fonctionnalité adaptée ;
3. de comprendre la documentation ;
4. de l'implémenter ;
5. de vérifier que son choix est pertinent.

Documentation :

<https://nextjs.org/docs>



36. Critères d'évaluation

L'évaluation ne portera pas uniquement sur le nombre de fonctionnalités réalisées.

Fonctionnalité

Le produit fonctionne-t-il réellement ?

Les principales fonctionnalités métier sont-elles utilisables ?

Architecture Next.js

Les mécanismes du framework sont-ils compris et correctement employés ?

Une attention particulière sera portée à la séparation Server / Client.

Qualité UX

Le produit est-il compréhensible et agréable à utiliser ?

Performance

La galerie reste-t-elle performante avec un nombre important de photographies ?

Responsive

L'application fonctionne-t-elle réellement sur smartphone ?

Robustesse

Les chargements, erreurs et cas particuliers sont-ils traités ?

Sécurité

Les zones privées et les données sensibles sont-elles correctement protégées ?

Qualité du code

Le code est-il structuré, maintenable et correctement typé ?

Design

Le résultat possède-t-il une identité visuelle cohérente et suffisamment aboutie ?

////////////////////////////////////

37. Niveau minimal attendu

À la fin du projet, le parcours suivant doit au minimum être possible :

1. Un photographe se connecte.
2. Il accède à son dashboard.
3. Il consulte ses clients.
4. Il consulte ses galeries.
5. Il ouvre une galerie et consulte ses photographies.
6. Un client reçoit l'URL d'une galerie.
7. Il saisit son code.
8. Il accède aux photographies.
9. Il sélectionne ses photographies préférées.
10. Sa sélection est enregistrée.
11. Il valide sa sélection.
12. Le photographe retrouve cette sélection depuis son dashboard.

Le produit devra proposer une expérience cohérente autour de ce parcours.

38. Philosophie du projet

Ce projet n'est pas un exercice consistant à reproduire une interface donnée.

Vous recevez :

- un besoin métier ;
- des données ;
- une API ;
- des contraintes ;
- un framework.

À vous de concevoir la solution.

Pendant tout le développement, posez-vous notamment cette question :

Est-ce que ce code a réellement besoin de s'exécuter dans le navigateur ?

Et plus généralement :

Pourquoi ai-je choisi cette architecture plutôt qu'une autre ?

Vous devrez être capables de défendre vos choix.

Livrable final

Vous devrez fournir :

- le dépôt Git du projet ;
- un README permettant de lancer l'application ;
- les variables d'environnement nécessaires documentées sans fournir de secret ;
- une application fonctionnelle ;
- une courte présentation du produit.

Lors de la présentation, vous devrez être capables d'expliquer quelques choix techniques importants réalisés pendant le développement.

Le résultat attendu n'est pas une succession d'exercices Next.js.

Le résultat attendu est un produit.